

I) Programmation dynamique

Dans les grandes lignes, la programmation dynamique est une méthode de programmation qui permet d'éviter de répéter certains calculs en stockant les résultats intermédiaires.

Les étapes pour résoudre un problème à l'aide de la programmation dynamique sont :

- décomposer les problèmes en sous-problèmes
- établir une relation de récurrence entre ces sous-problèmes
- résoudre les sous-problèmes dans l'ordre en utilisant la relation de récurrence, et en stockant le résultat de chaque sous-problème pour une potentielle utilisation future.

I.1) Exemple “simpliste” : suite de Fibonacci

La suite de Fibonacci est définie par :

$$\begin{cases} \text{fib}(0) = 1 \\ \text{fib}(1) = 1 \\ \text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2) \text{ pour } n \geq 2 \end{cases}$$

Si on utilise une fonction récursive simple pour calculer une telle suite, on tombe vite sur un problème.

```
def fib(n):  
    if n <= 1:  
        return 0  
    else:  
        return fib(n - 1) + fib(n - 2)
```

Cette fonction va faire 2^{n-1} appels récursifs !

La version en programmation dynamique consiste à calculer non plus $\text{fib}(n)$ directement, mais toute la suite en partant de 0, en stockant les résultats dans un tableau (une liste).

```
def fib(n):  
    resultats = [1, 1]  
    for i in range(2, n + 1):  
        resultats.append(resultats[-1] + resultats[-2])  
    return resultats[-1]
```

Cette fois, la complexité est en $O(n)$. C'est bien mieux !

I.2) Un vrai problème à résoudre en programmation dynamique : le rendu de monnaie

I.2.a) Exemples

Exemple 1

On a un système monétaire qui contient des coupures de 1, 2, 5 et 10. Combien de coupures, au minimum, pour rendre 17 ?

Réponse : on peut appliquer un algorithme “glouton” : on prend toujours la plus grande coupure qui ne dépasse pas le total. Ici, cela donne 10, 5, 2.

Il se trouve que c'est optimal.

Exemple 2

On prend un autre système monétaire, beaucoup moins pratique, avec des coupures de 1, 3 et 4, et on essaie de rendre 6. Cette fois, l'algorithme glouton renvoie 4, 1, 1. Ce n'est pas optimal ! On aurait pu utiliser 3 et 3.

I.2.b) Définition formelle et résolution de problème

Problème 3

On a n pièces $v_1 < v_2 < \dots < v_n \in \mathbb{N}$, et une somme à rendre $S \in \mathbb{N}$. On pose $v_1 = 1$.

On cherche un n -uplet $T = (x_1, x_2, \dots, x_n)$ tel que $S = \sum_{i=1}^n x_i v_i$, et qui minimise $\sum_{i=1}^n x_i$.

Solution en programmation dynamique 4

1. **sous-problèmes** : pour $1 \leq i \leq n, 1 \leq s \leq S$ on prend $m_i(s)$ le nombre minimal de coupures parmi $v_1, \dots, v_i$ pour arriver à une somme de s .

2. **relation de récurrence** :

- $\forall i \in \llbracket 1, n \rrbracket, m_i(1) = 1$
- $\forall s \in \llbracket 0, S \rrbracket, m_1(s) = s$
- sinon, $m_i(s) = \min \begin{cases} 1 + m_i(s - v_i) & \text{si } v_i < s \\ m_{i-1}(s) \end{cases}$ On utilise la pièce i
On n'utilise pas la pièce i

```
def rendu_monnaie(pieces, S):
    #initialisation du tableau résultat
    resultat = [[0]*S for _ in pieces]
    for i in range(len(pieces)):
        resultat[i][0] = 1

    for s in range(S):
        resultat[0][s] = s + 1

    #remplissage du tableau par s et i croissants
    for s in range(1, S):
        for i in range(1, len(pieces)):
            #cas ou la dernière pièce est trop grande
            if pieces[i] > s+1:
                resultat[i][s] = resultat[i-1][s]
            #cas général : on prend la dernière pièce ou non
            else:
                resultat[i][s] = min(1 + resultat[i][s - pieces[i]],
                                     resultat[i-1][s])

    return resultat[-1][-1]
```

Cette fonction a une complexité $O(nS)$ (la taille du tableau).

[lien vers un notebook pour executer ce code](#)

I.2.c) Retrouver la répartition

Comment peut-on alors retrouver la répartition des pièces ?

On peut suivre en remontant le chemin qu'a pris le résultat : si $1 + \text{resultat}[i][s - \text{pieces}[i]] < \text{resultat}[i-1][s]$, c'est qu'on a pris la pièce i , et inversement, jusqu'à arriver à un bord du tableau.

I.3) Un autre problème : la distance d'édition

Lorsqu'on veut mesurer la différence entre deux chaînes de caractères, il n'est pas toujours adapté de simplement comparer les lettres une à une : par exemple, "bonjour" et "onjour" sont des chaînes proches, mais n'ont aucune lettre en commun (et à la même position).

La distance d'édition est une solution à ce problème.

Problème 5

On définit trois opérations d'édits :

- **suppression** : on supprime un caractère dans la chaîne
- **ajout** : on ajoute un caractère dans la chaîne
- **modification** : on remplace un caractère par un autre

La distance d'édition entre deux chaînes s_1 et s_2 est le nombre minimal d'opérations d'édition pour passer de s_1 à s_2 .

Solution en programmation dynamique 6

1. **sous-problèmes** pour $0 \leq i \leq |s_1|, 0 \leq j \leq |s_2|$, on note $d(i, j)$ la distance d'édition entre $s_1[0 : i]$ et $s_2[0 : j]$

2. **relation de récurrence**

- $d(0, k) = d(k, 0) = k$
- $$d(i, j) = \min \begin{cases} 1 + d(i-1, j) & \text{(suppression)} \\ 1 + d(i, j-1) & \text{(ajout)} \\ d(i-1, j-1) + \mathbb{I}_{s_1[i] \neq s_2[j]} & \text{(modification)} \end{cases}$$

```
def distance_edition(s1, s2):
    """calcule la distance d'édition entre deux chaînes de caractère s1 et s2, par la
    méthode de la programmation dynamique."""
    #initialisation du tableau résultat
    resultat = [[0]*(len(s2) + 1) for _ in range(len(s1) + 1)]
    for k in range(len(s2) + 1):
        resultat[0][k] = k
    for k in range(len(s1) + 1):
        resultat[k][0] = k

    for i in range(1, len(s1) + 1):
        for j in range(1, len(s2) + 1):
            #calcul des trois cas
            supp = resultat[i-1][j] + 1
            ajout = resultat[i][j-1] + 1

            if s1[i-1] != s2[j-1]:
                modif = resultat[i-1][j-1] + 1
            else:
                modif = resultat[i-1][j-1]
            resultat[i][j] = min(supp, ajout, modif)

    return resultat[-1][-1]
```

[lien vers un notebook contenant le code](#)

I.4) Dernier problème : algorithme de Floyd-Warshall

Petit retour sur nos amis les graphes !

On va utiliser la programmation dynamique pour calculer le plus court chemin entre deux sommets d'un graphe.

On va prendre des contraintes un peu différentes (par rapport à Dijkstra) :

On cherche le plus court chemin entre chaque paire de sommets pour **tous** les sommets du graphe.

On se donne un graphe dirigé pondéré G avec un ensemble de sommets $S = \llbracket 1, n \rrbracket$ et un ensemble d'arrêtes $A \subset S \times S \times \mathbb{Z}$ (les arrêtes peuvent avoir un poids négatif !). On suppose que G n'a pas de cycles de poids négatif.

Soit la fonction $d(x, y)$ qui renvoie

- 0 si $x = y$
- w si x et y sont reliés par une arrête de poids w
- $+\infty$ sinon

(cela correspond à la matrice d'adjacence)

Solution en programmation dynamique 7

1. **découpage en sous-problèmes** : on note $\mathcal{C}(x, y, k)$ la taille du plus court chemin entre x et y deux sommets, en ne passant que par des sommets de $\llbracket 1, k \rrbracket$
2. **relation de récurrence**
 - $\mathcal{C}(x, y, 0) = d(x, y)$
 - $\mathcal{C}(x, y, k) = \min \begin{cases} \mathcal{C}(x, y, k-1) & \text{on ne passe pas par } k \\ \mathcal{C}(x, k, k-1) + \mathcal{C}(k, y, k-1) & \text{on passe par } k \end{cases}$

[lien vers un notebook contenant le code](#)

Note : on utilise uniquement la valeur de $\mathcal{C}(\cdot, \cdot, k-1)$ pour calculer $\mathcal{C}(\cdot, \cdot, k)$. Cela signifie que l'on est pas obligé de stocker tous les tableaux, on peut se contenter seulement du tableau précédent.

En revanche, si on veut reconstruire les chemins, on a besoin de tous les tableaux.